

Resource Allocation System

Iteration 30 – Minute-Based Time Slot Support

Najib Attar

May 2026

1 Iteration Goal

The goal of Iteration 30 is to improve the time model used by the resource allocation system. Before this iteration, each `TimeSlot` stored time using integer hour values only:

- `day`
- `startHour`
- `endHour`

This allowed requests such as 3-10-12, meaning Wednesday from 10:00 to 12:00. However, it did not support more precise bookings such as 10:00–10:30, 13:15–13:45, or 15:30–16:45.

This iteration changes the internal time representation to minute-level precision. The system can now represent shorter and more realistic scheduling intervals while preserving compatibility with the previous hour-only CSV format.

2 Changes from Previous Iteration

The previous iteration focused on improving multi-room exam allocation through the best-fit strategy. Iteration 30 does not add a new allocation strategy. Instead, it improves the core scheduling model used by all request types and allocation strategies.

The main changes are:

- Updated `TimeSlot` to store `startMinutes` and `endMinutes`.
- Preserved `getStartHour()` and `getEndHour()` for compatibility.
- Added minute-based getters such as `getStartMinutes()`, `getEndMinutes()`, `getStartMinute()`, and `getEndMinute()`.
- Updated request time parsing so `RequestFactory` supports both old and new time formats.

- Updated overlap checking to use minute-precise half-open intervals.
- Updated console, CSV export, and web display paths to show readable time values such as 10:30–11:00.
- Preserved existing request types, rules, allocation strategies, and CSV structure.

3 Scenario

A student, instructor, or staff member may need to reserve a room for less than a full hour. For example, a short meeting may only require 30 minutes:

1-10:00-10:30

This means Monday from 10:00 to 10:30. Another request may begin immediately after it:

1-10:30-11:00

These two requests should not conflict because the first one ends exactly when the second one starts. However, this request should conflict with the first one:

1-10:15-10:45

The new minute-based model allows the system to distinguish these cases correctly.

4 Modules and Their Tasks

Module	Task in Iteration 30
<code>TimeSlot</code>	Stores time using minutes from midnight instead of only integer hours. It still provides hour-based getters for older code paths.
<code>RequestFactory</code>	Parses both old hour-only <code>timeData</code> values and new minute-based values such as 3-10:30-12:00.
<code>AvailabilityRule</code>	Continues using <code>TimeSlot::overlapsWith()</code> , which now performs minute-precise overlap checking.
<code>OneTimeRequest</code>	Uses the improved <code>TimeSlot</code> model without changing its request structure.
<code>RecurringRequest</code>	Supports recurring minute-based time data such as 2-09:00-10:30;4-13:15-14:00.
<code>ExamRequest</code>	Uses the improved time model for exam time slots, including minute-based exam periods.

<code>AllocationWriter</code> and <code>RequestResultWriter</code>	Continue exporting the same CSV structures while displaying time values in readable <code>HH:MM</code> format.
Web Dashboard	Displays stored time values in a more readable form while leaving the backend CSV format unchanged.

5 Minute-Based TimeSlot Design

The updated `TimeSlot` model stores time as minutes from midnight:

- 10:00 is stored as 600.
- 10:30 is stored as 630.
- 13:45 is stored as 825.

The updated internal fields are:

Field	Meaning
<code>day</code>	Day number from 1 to 7.
<code>startMinutes</code>	Start time in minutes from midnight.
<code>endMinutes</code>	End time in minutes from midnight.

The old constructor style remains supported. A time slot created with start hour 10 and end hour 12 is internally stored as 600 and 720 minutes. This keeps older code and older CSV examples working.

The supported CSV time formats are:

Format	Example	Meaning
Old hour-only format	3-10-12	Wednesday 10:00–12:00
New minute format	3-10:30-12:00	Wednesday 10:30–12:00
Recurring minute format	2-09:00-10:30;4-13:15-14:00	Tuesday and Thursday precise slots

The overlap logic now uses half-open intervals:

`[start, end)`

This means the start time is included, but the end time is not. The overlap condition is:

`start1 < end2 && start2 < end1`

With this rule, 15:00–17:00 and 17:00–19:00 do not conflict. However, 15:00–17:00 and 16:30–18:00 do conflict.

6 Iteration Comparison

Iter.	Focus	Added / Modified	Prototype Effect
26	Request-type permission	<code>RequestTypeRule</code>	Separated request-type authorization from space-type authorization.
27	Allocation preparation	<code>assignedParticipants</code> and space pool access	Prepared the model and strategy interface for multi-room exam allocation.
28	Multi-room greedy	<code>MultiRoomExamGreedyStrategy</code>	Allowed splittable exams to use multiple classrooms from the room pool.
29	Multi-room best-fit	<code>MultiRoomExamBestFitStrategy</code>	Reduced wasted classroom capacity for splittable exams.
30	Minute-based time slots	Minute-based <code>TimeSlot</code> and updated time parsing	Allowed requests and exams to use precise time ranges such as 10:30–11:00.

7 Updated Class Diagram

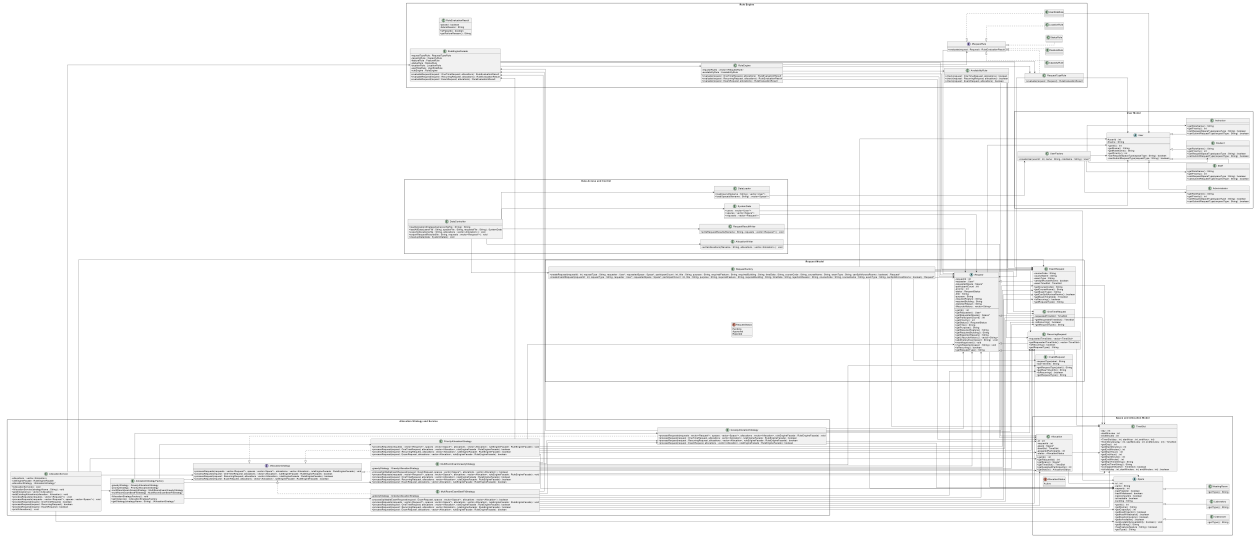


Figure 1: Updated class diagram after adding minute-based `TimeSlot` support.

The main class diagram change is in `TimeSlot`. The previous model stored `startHour` and `endHour`. The updated model stores `startMinutes` and `endMinutes`, while still exposing hour-based getters for compatibility.

8 Updated Workflow Diagram

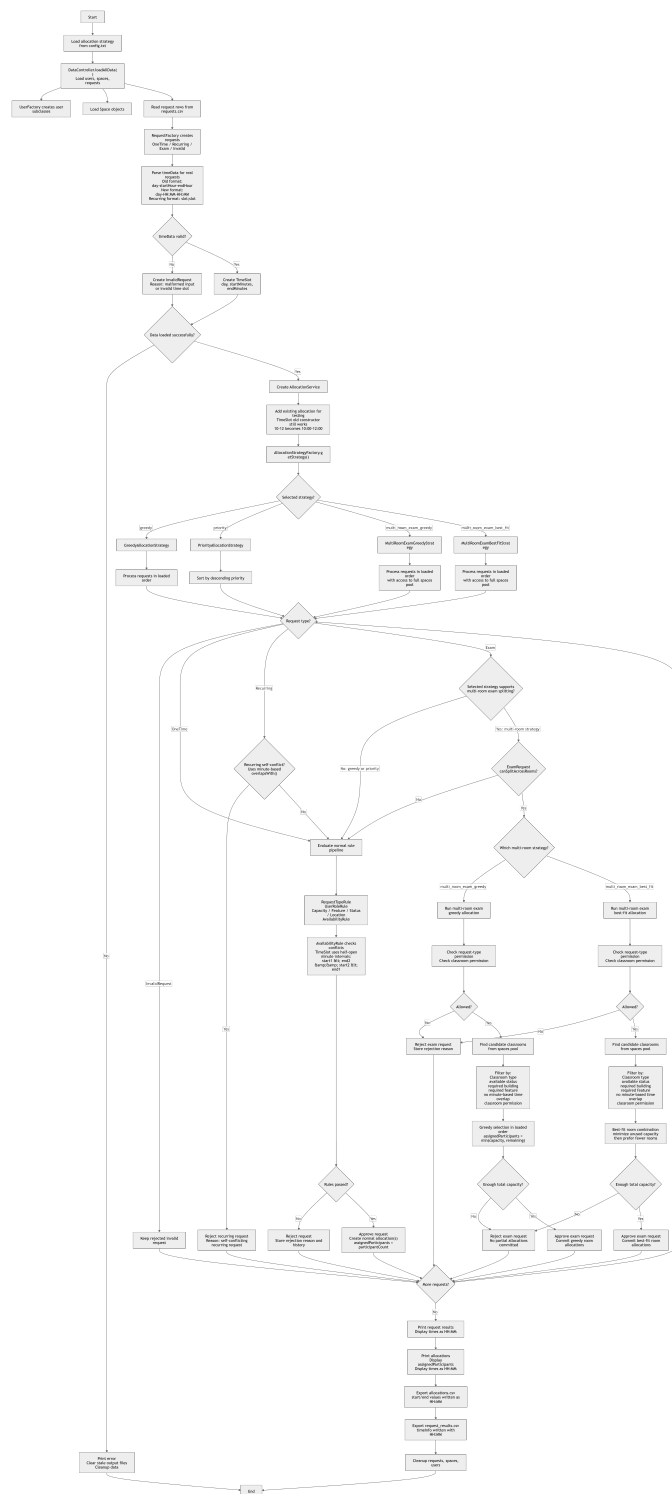


Figure 2: Updated workflow showing minute-based time parsing and overlap checking.

The workflow is mostly unchanged. Requests are still loaded from CSV files through `DataController`, and request objects are still created by `RequestFactory`. The important difference is that `RequestFactory` now parses time data into minute-based `TimeSlot` objects. Later, `AvailabilityRule` uses the updated `overlapsWith()` method for minute-accurate conflict detection.

9 Prototype Implementation

The prototype still supports the old CSV input style:

3-10-12

This is interpreted as Wednesday 10:00–12:00. The new format is also supported:

3-10:30-12:00

This is interpreted as Wednesday 10:30–12:00.

Recurring requests can also use minute-level time values:

2-09:00-10:30;4-13:15-14:00

Testing confirmed that adjacent requests do not conflict. For example, 10:00–10:30 and 10:30–11:00 can both be approved for the same room because their time intervals only touch at the boundary. A request from 10:15–10:45 conflicts with the first request because the intervals overlap.

The allocation strategies did not need algorithmic changes. Greedy, priority, multi-room greedy, and multi-room best-fit strategies continue to use the same request and allocation flow. They benefit from the improved `TimeSlot` model because availability checking is now more precise.

10 Summary

Iteration 30 improves the core scheduling model by changing `TimeSlot` from an hour-only representation to a minute-based representation. This allows the system to support realistic time ranges such as 10:00–10:30, 13:15–13:45, and 15:30–16:45.

The iteration preserves backward compatibility with the previous CSV format, so existing examples such as 3-10-12 remain valid. It also keeps request types, user permissions, allocation strategies, and CSV structure unchanged.

This iteration improves scheduling accuracy rather than allocation strategy behavior. The backend stores machine-friendly minute values, while the console and web dashboard show readable time values for users.